

CS 314H Programming Assignment #2: Text Generation

Project Overview

This project implements a character-level text generator. The program reads a source text, analyzes the statistical relationships between characters in that text, and then produces new random text that imitates the statistical properties of the source.

The central idea is the level of analysis, denoted as an integer k . A level k analysis records, for every sequence of k consecutive characters in the source (I refer to this as a seed), which characters are observed to follow that sequence and how often. Generation then proceeds one character at a time: the program looks up the current seed, chooses randomly among the characters that followed it in the source, adds that character, and slides the seed forward by one position. The hope is that with higher k , the more similar the generated text is to the English in the source.

The goals of the project were to:

- Implement the `TextProcessor` interface (`createProcessor`, `readText`, `writeText`) in a `RandomWriter` class.
- Provide a main method that validates four command line arguments and reports invalid input to `System.err`.
- Correctly reproduce the conditional character frequencies of the source, so that a character following a given seed is chosen with probability proportional to how often it follows that seed in the source.

Scope, Quality, and Limitations

Scope of the Program

The program accepts any plain text file as a source and any non-negative values of k and length. It correctly handles the extreme cases at both ends of the k range: $k = 0$ produces text drawn from the unconditional character distribution of the source, and large values of k produce long verbatim runs from the source. It handles a requested output length of zero by producing an empty file. It handles seeds that reach a dead end (a seed that occurs only at the very end of the source with no character after it) by getting a new random seed and continuing, as the specification requires.

The program also rejects any invalid command line input before any work is done. More specifically, wrong argument count, non-integer values for k or length, negative values, an unreadable source file, and a source file containing k or fewer characters all produce a message on `System.err` and terminate.

Quality

The implementation is correct on every test case described in **Testing**, including the corner cases. The more optimized lookup structure with HashMap makes generation fast enough where output length becomes the main bottleneck rather than source length, since building the analysis table is a single linear pass over the source, and each subsequent character costs is an on average $O(1)$ hash lookup.

Limitations

Memory could be an issue. The HashMap, “analysisTable”, stores one String key per position in the source, plus a Character object per position. This is the primary cost of optimization. For large sources combined with large k , memory consumption grows roughly as $O(n \cdot k)$ characters of key data. My original scanning approach used essentially no additional memory beyond the source string itself but was significantly slower.

When k is very large, the output is basically just copying the source file. When k approaches the length of the longest repeated substring in the source, most seeds have exactly one follower, and the output becomes an exact transcription of the source interrupted only by occasional reseeds. This is still correct but feels less meaningful with respect to true text generation.

The current algorithm has no word or sentence awareness. Since the analysis is purely character-based, the program has no concept of word boundaries, grammar, or sentence structure. At low k it freely produces non-words, and at all levels it can end pretty randomly mid-word.

The final clear limitation is file compatibility. I believe the program can only reasonably handle text files and would likely generate meaningless output with something like a binary encoded file.

Algorithm and Implementation

main(String[] args)

The main method validates the arguments by order of cost, with the cheapest first. The argument count is checked, then Integer.parseInt converts args[2] and args[3] inside a try block. Both k and length are then checked for non-negativity.

After the validation, main constructs a processor and call readText and writeText inside a try block that catches IOException. This catch handles both the unreadable-source case and the source-too-short case, since readText signals the latter by throwing IOException as well.

createProcessor(int level) and the constructor

createProcessor is a static factory that returns a new RandomWriter. The constructor is private, so the factory is the only way to obtain an instance. The constructor stores the level in a field.

readText(String inputFilename)

readText both reads the file into memory, and builds the analysis table.

Reading uses a BufferedReader wrapped around a FileReader, pulling one character at a time in a loop until read() returns -1. I found that BufferedReader matters for performance, without it, each read() call is a separate system call. Characters accumulate in a StringBuilder rather than a String, because Java strings are immutable and building an n-character string is unoptimal because you have to copy a new string every time, whereas StringBuilder maintains a resizable internal character array and appends in on average constant time.

After reading, the method verifies that the source is longer than k characters and throws IOException if not. If it is, analysisTable, a Map<String, List<Character>>, is created. For each index i from 0 up to fullText.length() - level, the k-character substring starting at i becomes a key, and the character at index i + level is appended to that key's list. The largest i visited is length - level - 1, so i + level is at most length - 1 and always in range.

computeIfAbsent creates the list on first sight of a key and returns the existing list on every subsequent sight, which unnecessary checks for pre-existing keys.

One thing to note is that the list of characters for simplicity literally contains each instance of a character following a seed rather than having some numeric data to represent that. It could be more memory efficient to do that, but this is easier to implement and makes random selection easier as well.

writeText(String outputFilename, int length)

This method itself mostly does nothing, as most of the generation is carried out by the private generate helper to produce the output string, then writes it to the file with a FileWriter in a try-with-resources block.

generate(int length) (helper)

The generation loop maintains a current seed and a StringBuilder for output, pre-sized to the requested length to avoid additional reallocations. The algorithm follows the following:

1. An initial seed is drawn by `randomSeed()`. Then for each of length iterations:
2. Look up the candidate next characters for the current seed.
3. If the candidate list is empty (dead end) draw a fresh random seed and look up again.
4. Choose uniformly at random from the candidates and append the chosen character to the output.
5. Slide the seed: drop its first character and append the chosen one.

Step 4 has a check for `level == 0`, because `"".substring(1)` throws `StringIndexOutOfBoundsException`.

`randomSeed()` (helper)

`randomSeed()` gets an index uniformly from `[0, n - k - 1]` using `Random.nextInt(fullText.length() - level)` and returns the `k`-character substring starting there. I have a designated constant field for a `Random` instance to call the function.

`getNextPossible(String seed)` (helper)

Returns `analysisTable.getDefault(seed, Collections.emptyList())`. The `getDefault` call supplies an immutable empty list for seeds absent from the table, which makes sense if a seed isn't present in the source.

The first working implementation, retained in the code as `getNextPossibleScan`, scanned the entire source on every call, testing each position against the seed and collecting the following character on a match. This required no additional memory but repeated the same scan for every output character, giving $O(\text{length} * n)$ total work. The table-based version front-loads a single $O(n)$ pass and reduces each lookup to constant time, trading memory for speed. I opted for the `HashMap` approach given memory isn't too much of a concern and the significant efficiency benefits.

Testing

This assignment is substantially harder to test than Assignment 1 for two reasons. First, the output is random, so no fixed input produces a fixed expected output. Second, the interface is file-based, so the natural testing entry points require filesystem setup and teardown.

I tried to address both directly. The file-based problem was solved with JUnit's `@TempDir`, which provides a fresh temporary directory per test and deletes it afterward. A `sourceFile` helper writes a string to a temp file and returns its path, so test cases can be written as

plain string literals while still using the real file-reading code path. The randomness problem was addressed by constructing tests of three types, described below.

Since `generate()` takes a length and returns a string, touching no files, I opted for most of the substantive tests to target `generate()` and `getNextPossible()` directly rather than going through the file interface. This is also why `generate()` and `getNextPossible()` aren't private, so they can be reached by the test file.

Type 1 (Tests with a single possible correct output)

`testUniformSource` uses the source "aaaaaaaa" at $k = 2$. Every seed is aa and every follower is a, so the output is fully determined and can be compared with `assertEquals` against a literal string. This catches obvious failures in the generation loop.

`testUniqueSeededSource` does something similar. The source "abcabcabcabc" at $k = 2$ has exactly three distinct seeds, and each has exactly one distinct follower: ab has c, bc has a, ca has b. The source is also long enough that no dead end is reachable. The output must thus be a continuation of the abc cycle, which the test verifies character by character. If it breaks, we know exactly where from the `assertEquals` message

Type 2 (Statistical verification)

`testCorrectCharacterFrequencies` uses the source "aab" at $k = 0$, where the expected distribution is exactly $2/3$ for a and $1/3$ for b. Generating 6,000 characters and counting occurrences of a should give a value near 4,000; the standard deviation of this binomial is approximately 36.5, and the test accepts anything within roughly five standard deviations.

This test is probabilistic and can in principle fail on correct code, though it's extremely unlikely.

Type 3 (Properties that have to be true for every random draw)

`testWriteTextCorrectLength` asserts that requesting 250 characters produces a file of exactly 250 characters. `testKEqualsZero` asserts that every character in a 500-character output at $k = 0$ appears somewhere in the three-character source. `testDeadEnd` asserts only that generation completes without throwing and produces the right length.

Corner cases and why each was chosen

`testReadTextRejectsKCharacterSource` verifies the specification that requires the source to contain more than k characters. A source of exactly k is the single input that distinguishes a correct $\leq$ check from an incorrect $<$ check.

testKEqualsZero checks the case where the seed is the empty string. This exposes the `"".substring(1)` hazard in the sliding step. The test asserts both that no exception is thrown and that the output characters are drawn from the source.

testZeroLengthProducesEmptyFile asserts not only that the content is empty but that the file was actually created, as a program that returned early without opening the output file would produce different observable behavior.

testOverlappingSeeds checks for behavior of adjacent identical seeds. The source "aaab" at $k = 2$ contains the seed aa at index 0, followed by a, and again at index 1, followed by b. Both must be counted. This case specifically targets the error of advancing a scan by k positions after a match instead of by 1, which would silently drop the second occurrence and skew all subsequent probabilities.

testDeadEnd tests for seeds that have no following character. The source "abcd" at $k = 2$ makes the seed cd reachable by sliding, and cd has no follower. Without correct reseeding this throws.

testUnknownSeed tests for seeds that don't exist in the source. Querying "zz" against a source containing no such substring must return an empty list rather than null, which is what allows the caller's `isEmpty()` check to work.

testReadTextThrowsOnMissingFile confirms the `IOException` is thrown when invalid file input is given.

Interesting Results

Result 1: I could obtain the characters with the correct probability just from the list itself, with no arithmetic. The design that retains duplicate followers in each list means the program never computes a probability, yet it samples exactly from the correct conditional distribution. Selecting uniformly at random from [e, e, e, r] is precisely a $3/4$ chance of e. This made it much simpler to implement, although perhaps a bit less performant compared to a list that contained numeric data referring to the frequency.

Result 2: The transition from nonsense to plagiarism is relatively narrow. Increasing k does not improve output quality indefinitely. At low k the output is unpronounceable, in a middle interval it produces convincing word-like text, and past a certain point it simply reproduces long verbatim stretches of the source. I hypothesize the width of that middle interval depends on the source. Short and highly repetitive text like a Dr. Seuss book exhausts its distinct seeds at a much lower k than a full-length play does, so it degenerates into copying far sooner.

K = 3, MuchAdo (nonsense)

at hown crome; an th a ch, shat, God me thurothiess ansink thim annot wilessest and
Heart-thatron in. It And Godeadgme, I will noth. Liver; mor surestelignionave thy, melike a
God of its. WATO.

BEATR

K = 3, OneFishTwoFish (somewhat recognizable English)

not one, fun is cow?

Hop!

Bump

with they has is one somethings a will I book why.

He like

my ear.

SO...

if your can sing

if you make three.

☐ hot know

my has fun

to far

five, I sever

from this one.

Assumptions

- The source file fits in memory. The entire text is held as a single String, and the analysis table adds further memory proportional to the source length. Sources of a few megabytes are fine; a source of many gigabytes is not supported.
- The source is plain text. Other types would technically be read, but the results are meaningless.
- All characters are significant. Newlines, tabs, and multiple consecutive spaces are treated as ordinary characters and participate in exactly like letters. No cleaning is

performed, which is why generated output preserves the line-break patterns of the source.

- The initial seed may start anywhere. The specification calls for a random initial seed of k consecutive characters and does not require it to align with a word or sentence boundary, so output commonly begins in the middle of a word.

Problems Encountered

The provided skeleton contained a private constructor with a comment to add logic, and the parameter name `level` shadowed the field of the same name. Because the field was never assigned, it silently kept its default value of 0 regardless of the k supplied on the command line. The error was a `StringIndexOutOfBoundsException: String index out of range: -1` thrown from the seed-sliding line.

The stack trace was misleading, because it pointed at the sliding logic rather than at the constructor. Debugging came down to reading the reported index: -1 is not an index that could plausibly arise from the arithmetic in that line unless the seed was empty, and the seed could only be empty if the `level` were 0 (which it should not have been for the k that had been passed on the command line). Printing the field's value at the top of `generate` confirmed it. The fix was just `this.level = level`.

Another error happened in the early version of the `getNextPossible` code which cut the candidate substring into a local variable `t` and then called `t.charAt(i + level)` to retrieve the following character. But `t` holds only k characters, while $i + level$ is an index into the full text. This threw immediately on the first match. The fix was `fullText.charAt(i + level)`.

When switching to the table, `getNextPossible` still declared a return type of `ArrayList<Character>`, but `Map.getOrDefault` returns the map's value type, `List<Character>`. The compiler reported a confusing error about `Collections.emptyList()` producing `List<Object>`. The fix was to change the declared return type to `List<Character>`, which both resolved the conversion and gave `emptyList()` a target type to infer from. This made me realize it's usually better to reference the interface than the implementation for compatibility.

Social Impact

Generative models such as `RandomWriter` and LLMs have the potential to revolutionize society through their contributions to communication and productivity while posing several ethical and societal risks. In terms of benefits, these models can automate writing tasks, aid in creative processes, and make advanced communication technologies accessible to everyone. At the same time, the capability of these models to create convincingly realistic

text gives an opportunity for ill-intent individuals to create extensive misinformation campaigns. In addition to that, the training of these models on vast amounts of information available online puts them at risk of reproducing and amplifying socially prejudiced beliefs, offensive stereotypes, and false claims. Thus, serious risks involved explain why organizations might restrict public access to the advanced models until security mechanisms and regulations can be put in place.

Sources:

<https://www.ibm.com/think/insights/10-ai-dangers-and-risks-and-how-to-manage-them>

<https://safe.ai/ai-risk>

Code specific

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/StringBuilder.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

<https://docs.oracle.com/javase/8/docs/api/java/util/Random.html>

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/HashMap.html>

<https://docs.junit.org/5.5.0/user-guide/#writing-tests>